

Pentesting

Penetration Test Report

Target: Hackthissite.org
Date of test: 12/23/2025

Contents

Executive summary.....	4
1.1 Overview.....	4
1.2 Scope.....	4
Basic 1.....	4
1.1 Vulnerability description.....	4
1.2 Key findings.....	5
1.3 Security recommendations.....	5
Basic 2.....	5
2.1 Vulnerability description.....	5
2.2 Key findings.....	5
2.3 Security recommendations.....	5
Basic 3.....	6
3.1 Vulnerability description.....	6
3.2 Key findings.....	6
3.3 Security recommendations.....	6
Basic 4.....	6
4.1 Vulnerability description.....	6
4.2 Key findings.....	7
4.3 Security recommendations.....	7
Basic 5.....	7
5.1 Vulnerability description.....	7
5.2 Key findings.....	7
5.3 Security recommendations.....	8
Basic 6.....	8
6.1 Vulnerability description.....	8
6.2 Key findings.....	8
6.3 Security recommendations.....	8
Basic 7.....	9
7.1 Vulnerability description.....	9
7.2 Key findings.....	9
7.3 Security recommendations.....	9
Basic 8.....	9
8.1 Vulnerability description.....	9
8.2 Key findings.....	10
8.3 Security recommendations.....	10
Basic 9.....	10
9.1 Vulnerability description.....	10
9.2 Key findings.....	10
9.3 Security recommendations.....	10

Basic 10..... 11
 10.1 Vulnerability description..... 11
 10.2 Key findings..... 11
 10.3 Security recommendations..... 11
Basic 11.....11
 11.1 Vulnerability description..... 11
 11.2 Key findings..... 11
 11.3 Security recommendations..... 12
Conclusion..... 12

Executive summary

The aim of this penetration test is to conduct penetration testing to identify vulnerabilities within the hackthissite website. Providing a practical demonstration of how a malicious actor may breach the site. Leading to the loss of availability, integrity, and confidentiality. All testing was done and performed in accordance with the website's terms of service and ethical guidelines. This report contains the vulnerabilities, key findings, and security recommendations for the 11 levels of the “Basic” web challenges to be reviewed and possibly reproduce the test.

1.1 Overview

The environment contains numerous vulnerabilities, such as passwords located in the HTML, easily redirect password files, command and server-side injection capability, cookies with accessibility value, weak encryption, and unprotected Apache files. Which makes them susceptible to data breaches and system takeovers.

1.2 Scope

The following information were included in the scope of the penetration test:

- Hackthissite's web pages for basic 1 to 11 challenges
- Network level, technical penetration testing against internet facing hosts
- Intending to simulate the actions of a malicious actor who attempted to obtain the password within the webpages
- The penetration test was carried out from one Windows PC using the Brave browser

Basic 1

1.1 Vulnerability description

The basic level 1 website's vulnerability comes from the HTML code containing the password necessary to log in. When right-clicking the website and clicking 'view page source,' the password is within the comments section above the input form section.

1.2 Key findings

- A malicious actor can gain access to the system and potentially access sensitive files to alter the website by using the password written directly in the HTML code.

-

```
<b>Level 1(the idiot test)</b>
</center><br /><br />
This level is what we call "The Idiot Test", if you can't complete it, don't give up on learning all you can, but, don't go begging to
<!-- the first few levels are extremely easy: password is 4464a1af -->
<center><b>password:</b><br /><form action="/missions/basic/1/index.php" method="post"><input type="password" name="password" /><br />
  <center><table border="0" width="80%" cellspacing="0" cellpadding="0">
    <tr>
      <td class="dark-td">&nbsp;<b>Help!</b></td>
      <td class="light-td">&nbsp;<br /><td>If you have no idea what to do, you must <a href="http://www.w3schools.com/HTML">learn HTML
    </tr>
  </table></center></td>
```

1.3 Security recommendations

To prevent malicious actors from obtaining the password, it is recommended to remove the password from the comments within the website's HTML. Anyone with basic knowledge of HTML and how to view the website's page source, would find the password. Instead, hash and salt the password file, store in a file unaccessible from the directory.

Basic 2

2.1 Vulnerability description

Basic 2's vulnerability comes from the network security team not uploading the password file that would be used to compare the user's input to. With nothing to compare to, it allows attackers to click submit, and it grants them access. Given, the user's input would be compared to null instead, making it, so no password needs to be typed in to gain access.

2.2 Key findings

- A malicious actor can gain access without any difficulties or additional technical knowledge.
- Not entering a password and clicking submit grants access

2.3 Security recommendations

Having the network security team upload the password file to the system to compare the user's input to and ensuring it isn't easily accessible by an attacker. Testing before implementing the change is suggested.

Basic 3

3.1 Vulnerability description

In basic 3, the network security team uploaded the password file that would be used to compare the user's input to. Upon inspecting the website's page source, the HTML has the password file name and adding it to the end of the web address redirects to the file where the password is displayed.

3.2 Key findings

- Accessing the system and view the contents of the password file by using the file location with some basic HTML knowledge and viewing the site's page source. Adding "password.php" to the URL is enough to view the password.

-

```
<center><div style="width:80%"><div class="dark-td"><h2>Notice</h2></div><div class="
    <form action="/missions/basic/3/index.php" method="post">
    <input type="hidden" name="file" value="password.php" />
    <input type="password" name="password" /><br /><br />
    <input type="submit" value="submit" /></form>

</td>
```

```
line wrap
607357ea
```

3.3 Security recommendations

To prevent attackers from accessing the password.php file, the file would need to be encrypted, and the HTML would need to set the password file exclusively as an included password. Preventing it from being accessed directly by adding it to the URL.

Basic 4

4.1 Vulnerability description

Basic 4's page, the password, can be emailed directly to a member of the network security team. Redirecting it is possible by using developer tools to edit the page's source HTML to replace the email, have the password be sent to the new email, and having access to the site with said email.

4.2 Key findings

- It's simple to edit the website's source to alter the email and gain access when clicking the button to send the email. With some basic HTML knowledge, viewing the site's page source, and usage of the 'document.designMode' feature.

•



4.3 Security recommendations

Removing the email from the client side and moving said code to the server side to prevent it from being edited and misused by a third party. Sensitive information should not be hardcoded into the client side as it can be manipulated easily.

Basic 5

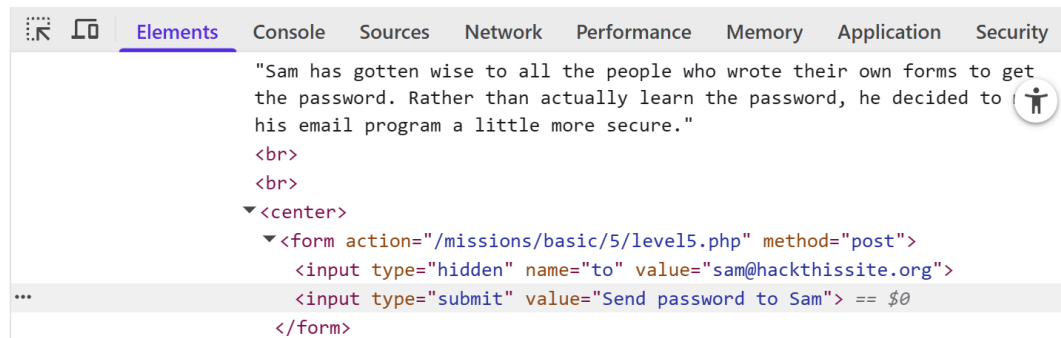
5.1 Vulnerability description

In Basic 5, the previous email is not as easily accessible, with adjustments made to the HTML making it hidden from being edited in developer mode. Using the inspect element option on the button makes it visible, accessible to edit, and redirect the email.

5.2 Key findings

- Similar to basic 4, a malicious actor can edit the website's source to alter the email and gain access when clicking the button to send the email. Using basic HTML knowledge, viewing the site's page source, inspecting the desired button, and usage of the 'document.designMode' feature, is it possible to obtain the password

- Evidence:



5.3 Security recommendations

Removing the email from the HTML and using a secure password manager would be a more effective solution, than attempting to hide the input value for the email to the network security team.

Basic 6

6.1 Vulnerability description

In Basic 6, the encrypted password was obtained and information that the encryption key is publicly available is sufficient to test out what encryption was used after using the encryption to find possible algorithms.

6.2 Key findings

- Unlike the previous pages, either brute force the encryption or test out a few basic examples to identify the algorithm used to decode the found password.
- 11111111 produces Your encrypted string is: '12345678' showing it is using the ASCII algorithm. Where it's 1+0, 1+1, 1+2, 1+3, 1+4, 1+5, 1+6, 1+7. To decrypt its subtraction according to the previous pattern and look through the ASCII table for its corresponding value.

6.3 Security recommendations

Removing the email from the HTML and shifting it to the server side rather than attempting to hide the input value for the email to the network security team. Sensitive information should not be coded in the client side as it can be edited and manipulated by a third party.

Basic 7

7.1 Vulnerability description

In Basic 7, the unencrypted password is in the directory and accessible using a UNIX script input section in the site. The script doesn't sanitize the input, allowing an attacker to use command injection, to list the context of the directory. Then open said file containing the password to gain access.

7.2 Key findings

- A malicious actor could brute force figuring out what files or view the index by using the command injection “;ls” command to list out the current directory.

```

      January 2026
Mon Tue Wed Thu Fri Sat Sun
           1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31
```

```

index.php
level7.php
cal.pl
.
..
k1kh31b1n55h.php
```

7.3 Security recommendations

To prevent the use of command injection, the user input needs to be sanitized. Assuming all input could be malicious ensures the security of the site before the input is executed.

Basic 8

8.1 Vulnerability description

In Basic 8's page, the unencrypted password is in the directory and accessible using the input section within the site page. The main difference is the script does sanitize the input to a certain degree. It prevents command injections, but not server-side injection. Allowing for an attacker to see the directory, locate the password file, and obtain the password. Using the “<!--#exec cmd="ls ../" -->”.

8.2 Key findings

- Typing the server-side injection command “<!--#exec cmd="ls ../" -->” lists out the directory and password file.

Hi, au12ha39vc.php index.php level8.php tmp! Your name contains 39 characters.

8.3 Security recommendations

Avoid using user input on pages that require server-side directives will lower the probability of a successful server-side injection attack. Website pages using .stm, .shtm, and .shtml make it easier for a malicious actor to identify the server is vulnerable to SSI injection, avoiding using such pages is ideal. The golden rule is not to trust user’s input and sanitize it.

Basic 9

9.1 Vulnerability description

In Basic 9’s page, there is no user input besides the password section, it does sanitize the input, so command injection and server-side injections are blocked. To see the directory, locate the password file, and obtain the password, is possible when returning to the previous page and using server-side injection to view basic 9’s directory.

9.2 Key findings

- Returning to the previous level to utilize the user input and the “<!--#exec cmd="ls ../../9/" -->” lists out the directory and password file.

Hi, index.php p91e283zc3.php! Your name contains 24 characters.

-

9.3 Security recommendations

Similar to basic 8’s recommendations, not using user input on pages that require server-side directives, not using .stm, .shtm, and .shtml pages, not to trust user’s input, and sanitize it before implementing it. Preventing basic 8’s user input from accepting server-side injections to go up a level in the directory to basic 9’s directory, and hashing and salting the password file.

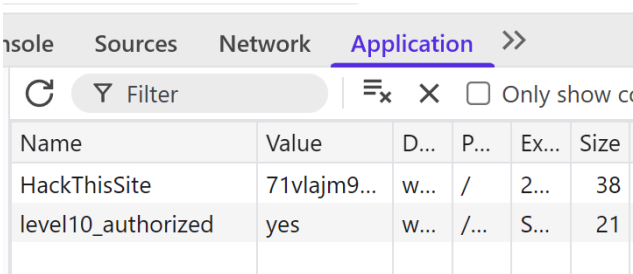
Basic 10

10.1 Vulnerability description

Using the developer’s tools to view the cookies in the application section for basic 10, by doing so, displays the authorization to enter the site. With usage of the ‘document.designMode’ feature, changing the value of the cookie grants access.

10.2 Key findings

- Editing the cookie to allow access to the application section without much difficulty. Leading
-



The screenshot shows the Chrome DevTools 'Application' tab. The 'Cookies' section is expanded, displaying a table of cookies. The table has columns for Name, Value, Domain, Path, Expires, and Size. Two cookies are listed: 'HackThisSite' with value '71vlajm9...' and 'level10_authorized' with value 'yes'.

Name	Value	D...	P...	Ex...	Size
HackThisSite	71vlajm9...	w...	/	2...	38
level10_authorized	yes	w...	/...	S...	21

10.3 Security recommendations

Removing the cookie and utilizing the means of a hashed and salted password file in an encrypted file, instead of having a cookie providing access to the site. Cookies can maintain the authentication state after logging in, but not as an authentication, the server should be used to identify and grant access to the site.

Basic 11

11.1 Vulnerability description

In basic 11’s page the directory listening is enabled, allowing malicious actors to find the directory where the password file would be. In the directory, there is no protection against the .htaccess Apache command, displaying the password file. Then used on the index.php to input the password.

11.2 Key findings

- Using the .htaccess command display the directory files, making it easier to identify and locate the password file.
-

```
IndexIgnore DaAnswer.* .htaccess
<Files .htaccess>
require all granted
</Files>
```

11.3 Security recommendations

Adjusting the file permissions is necessary when using Apache to prevent malicious actors from using the .htaccess command to display files and the directory is moved to the server side to avoid it potentially being accessed by a third party.

Conclusion

The security issues found on the 1 to 11 basic webpages are very severe and do not require a great deal of effort to exploit, the fixes are simple and easy to implement. It is recommended, to hash and salt the password files, and shift it to a private server along with a tightly controlled SSH server. Preventing malicious attackers from using command and server-side injections or remote attacks to the website. There could be a rise of SSH attacks, but it is easier to manage and mitigate. If the vulnerabilities are not resolved, it could lead to the loss of availability, integrity, and confidentiality.